

Browser tabs: dqn full form - Google, Traffic Management S, Untitled14, Home, Untitled13, Home, Ask Gemini

Address bar: localhost:8888/notebooks/Untitled13.ipynb?

JupyterLab interface: Untitled13 Last Checkpoint: 48 minutes ago

Menu: File Edit View Run Kernel Settings Help

Buttons: +, -, Copy, Paste, Run, Stop, Code

Right side: JupyterLab, Python [conda env:base], Anaconda Toolbox

```
[2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import ttest_rel

np.random.seed(42)

# Create synthetic dataset
n = 100
df = pd.DataFrame({
    "Demand": np.random.randint(50,200,n),
    "Holding_Cost": np.random.uniform(2,8,n),
    "Ordering_Cost": np.random.uniform(40,120,n),
    "Shortage_Cost": np.random.uniform(15,35,n),
    "Lead_Time": np.random.randint(1,8,n),
    "Current_Stock": np.random.randint(20,180,n)
})

# Demo Dynamic Programming processed values
dp = pd.DataFrame()
dp["Demand"] = df["Demand"]*0.95
dp["Holding_Cost"] = df["Holding_Cost"]*0.90
dp["Ordering_Cost"] = df["Ordering_Cost"]*0.92
dp["Shortage_Cost"] = df["Shortage_Cost"]*0.85
dp["Lead_Time"] = np.maximum(df["Lead_Time"]-1,1)
dp["Current_Stock"] = df["Current_Stock"]*1.05

# Demo Monte Carlo processed values
mc = pd.DataFrame()
mc["Demand"] = df["Demand"] + np.random.normal(0,8,n)
mc["Holding_Cost"] = df["Holding_Cost"] + np.random.normal(0,0.4,n)
mc["Ordering_Cost"] = df["Ordering_Cost"] + np.random.normal(0.6,n)
```

System tray: 31°C Mostly cloudy, Search, Taskbar icons, ENG INTL, 11:11 03-08-2026

localhost:8888/notebooks/Untitled13.ipynb?

Jupyter

Untitled13 Last Checkpoint: 48 minutes ago

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python [conda env:base] Anaconda Toolbox

```
)
print("\nPERFORMANCE COMPARISON\n")
print(performance)

# Statistical significance
rows=[]
for c in df.columns:
    t,p=sttest_rel(dp[c],mc[c])
    rows.append([c,round(t,3),round(p,5),"Significant" if p<0.05 else "Not Significant"])

stats=pd.DataFrame(rows,columns=["Feature","T-Statistic","P-Value","Result"])
print("\nSTATISTICAL SIGNIFICANCE\n")
print(stats)

# Overall evaluation
dp_win=(performance["Better Method"]=="Dynamic Programming").sum()
mc_win=(performance["Better Method"]=="Monte Carlo").sum()

print("\nOVERALL EVALUATION")
print("Dynamic Programming Better:",dp_win)
print("Monte Carlo Better:",mc_win)
if dp_win>mc_win:
    print("Overall Winner: Dynamic Programming")
elif mc_win>dp_win:
    print("Overall Winner: Monte Carlo")
else:
    print("Overall Result: Tie")

# Graph
x=np.arange(len(df.columns))
w=0.35
plt.figure(figsize=(10,5))
plt.bar(x-w/2,summary["DP Mean"],w,label="Dynamic Programming")
plt.bar(x+w/2,summary["MC Mean"],w,label="Monte Carlo")
```

31°C Mostly cloudy

Search

ENG INTL 11:12 03-08-2026

